

9) Frontend User Experience

Last updated by | Mennell, Jacob | 19 May 2026 at 14:32 BST

Frontend User Experience

Purpose

This page describes the browser-based user experience provided by the solution and how the frontend presents claim data, extracted information, checks, feedback, and administrative functions.

Scope

The frontend is a React application using route-based navigation and authenticated access patterns. It provides a claim-centric user experience for claims handlers, reviewers, and administrators.

Application Overview

- Application name: Claims Augmented Triage
- Purpose: support claims adjusters and related operations users with claim review, AI-assisted outputs, and workflow interaction
- Owning team: Claims
- Access model: Microsoft Entra ID SSO using MSAL with role-based access control

Frontend Environments

- Dev: [Claims Triage AI \(MVP\) Dev](#)
- Test: [Claims Triage AI \(MVP\) Test](#)
- UAT: [Claims Triage AI \(MVP\) UAT](#)

The frontend is delivered as a browser-based claims application rather than a standalone document viewer. Users move through claim summaries, claim details, AI-assisted outputs, and feedback activities within one consistent experience.

Users And Access

The application supports two primary user roles:

- User
- Admin

Users authenticate through Microsoft Entra ID single sign-on. Once authenticated, the frontend acquires tokens for backend API access and uses role-based access control to determine which routes and functions are available.

Administrative functions are protected behind explicit route guards. Access to the application is managed outside the frontend itself, with user onboarding handled through the organization's access-request process.

User Experience Model

The route structure points to several core user journeys:

- signing in through Microsoft Entra ID

- landing on a claims overview page
- navigating into an individual claim and its model-specific view
- reviewing claim details and first-notice-of-loss content
- reviewing extracted information for a claim
- reviewing routing checks and AI-assisted checks
- submitting feedback on claim-level AI outputs when the user is the claim owner
- accessing administrative pages when the user has sufficient privileges

The frontend is designed around claims as the primary navigation object, with extracted insights and checks presented as related views of the same claim context.

Key Features

Entra SSO login

- Description: Users authenticate through Microsoft Entra ID and the frontend acquires tokens for API calls.
- User roles: User, Admin
- Related backend service: Authentication through Entra ID and API token validation

View claim summaries

- Description: Users can browse a summary view of claims from the landing page.
- User roles: User, Admin
- Related backend service: Claims API

View claim details

- Description: Users can open a claim and review detailed claim information.
- User roles: User, Admin
- Related backend service: Claims API

Assign claim owner

- Description: Users can assign or update claim ownership from the claim experience.
- User roles: User, Admin
- Related backend service: Claims API

View extracted information

- Description: Users can inspect structured outputs and linked evidence sources.
- User roles: User, Admin
- Related backend service: Claims API

Review routing and AI checks

- Description: Users can review check results, evidence, and supporting explanations.
- User roles: User, Admin
- Related backend service: Claims API

Submit feedback

- Description: Claim owners can submit feedback on AI outputs and routing recommendations.
- User roles: Claim owner
- Related backend service: Claims API

Admin functions

- Description: Admin users can access privileged pages and operational functions.
- User roles: Admin
- Related backend service: Claims API

Main Pages

Home page

The home page presents a claims overview experience. Based on the page code, it includes:

- date range filtering
- role-aware user filtering
- paged and sortable grid views
- row-based navigation into individual claim experiences

This acts as the operational entry point for day-to-day claim browsing.

Claim overview pages

The route structure includes a claim layout and nested claim pages. This indicates a shared claim context where multiple views can reuse the same underlying claim data.

Within an individual claim, users can:

- review detailed claim information
- assign or update the claim owner
- inspect first-notice-of-loss data
- move into extraction, checks, and model-specific views

Information Extracted page

The extracted information page presents structured outputs derived from the claim. It also supports source grouping and source inspection behavior, allowing the user to see where information came from and to navigate associated documents.

The code shows that extracted values are rendered alongside grouped source references, which is important for trust and traceability.

Routing and Generic Check pages

The generic check page presents check outcomes, supporting evidence, and associated feedback interactions. It appears to support both page-level and popup-style usage depending on how the check path is supplied.

This allows checks to function as both standalone review experiences and embedded review components.

The wider frontend context also includes routing-check views for individual claims. These provide a practical way to review claim routing outputs and related supporting evidence.

AI insights and feedback

The frontend supports AI insight review for scenarios such as:

- policy coverage checks
- claims of interest checks
- subrogation checks
- risk of escalation into litigation
- ongoing litigation checks

Feedback is integrated into these experiences so the frontend does not only display outputs; it also captures user evaluation of those outputs.

Admin page

Administrative functionality is routed separately and protected by an admin-only route guard, indicating a clear distinction between standard operational usage and privileged configuration or access management tasks.

The admin experience also supports operational actions such as triggering job execution flows for specific claims.

User Workflows

Typical user workflows include:

1. Sign in through Microsoft Entra ID.
2. Review the main claims table.
3. Open an individual claim.
4. Review claim details and first-notice-of-loss information.
5. Assign or update the claim owner when required.
6. Review extracted information, routing checks, and AI insights.
7. Submit feedback on AI outputs when the user is the claim owner.

Admin users also have a separate operational workflow through the admin page, where they can trigger job execution endpoints for individual claims when needed.

Feedback And Ownership Rules

The frontend includes explicit feedback handling for claim outputs, with feedback rights tied to claim ownership.

- Routing checks include a focused feedback question on whether the AI outcome was conclusive.
- AI insights include feedback on whether the AI output was conclusive and whether a leakage opportunity exists.
- If a user identifies a leakage opportunity, the UI can collect follow-up detail.
- If a user does not identify a leakage opportunity, the UI can require a reason for that decision.

This is an important design choice because it allows the application to capture operational review signals directly in the same interface where AI-supported outputs are consumed.

Frontend Architecture At A High Level

The application uses:

- React components and routed pages
- client-side navigation with React Router
- context-based state for user and claim data
- API service wrappers for backend communication
- reusable source-viewing and document-viewing components

This is a practical structure for a claims review interface where several pages rely on shared claim state and common backend services.

Relationship To Backend Services

The frontend depends heavily on the API layer for:

- claim summaries
- claim details
- claim documents
- extracted outputs
- check data
- feedback submission and retrieval
- orchestration actions exposed through admin-facing workflows

The UI does not appear to call workers directly. Instead, it relies on the API to provide a stable boundary over persisted data and workflow outputs.

The primary backend integration surface is the Claims API.

User Experience Characteristics

At a high level, the UI is designed with several practical characteristics:

- claim-centric navigation rather than document-centric navigation
- traceability through linked sources and document references
- support for large operational lists using grids, paging, and filtering
- role-sensitive access to administrative functions
- reuse of common components for source display and document rendering

The experience also balances operational scale and detailed review. Users can move from a broad claims list into a narrow evidence-backed investigation flow without leaving the application.

Frontend Interaction Diagram



flowchart TD

```

SIGNIN[Entra SSO Sign-In] --> HOME[Claims Overview]
HOME --> CLAIM[Claim Context]
CLAIM --> FNOL[FNOL Details]
CLAIM --> INFO[Information Extracted]
CLAIM --> CHECK[Routing and Generic Checks]
CLAIM --> MODEL[Model-Specific Page]
ADMIN[Admin] --> MODELS[Model Access Management]
ADMIN --> JOBS[Run Claim Jobs]
INFO --> SOURCES[Source and Document Inspection]
CHECK --> FEEDBACK[Feedback Capture]

```

Representative Screens

The frontend supports a consistent set of operational and review screens, including:

- a home screen showing the claims table
- an individual claim overview page
- first-notice-of-loss detail popups within a claim
- information extraction views for individual claims
- routing-check popups and full routing-check pages
- AI insight panels for specific review scenarios
- an admin page for privileged functions

Screenshots of these flows are useful in the target wiki or presentation environment, but they are best managed there rather than embedded through temporary workspace-specific attachment paths in this repository.

Design Decisions

- The UI is organized around claims because that is the natural unit of user work.
- Source visibility is built into extracted information and checks, which supports explainability and reviewer confidence.
- Administrative capability is isolated behind explicit role checks.
- The frontend depends on backend APIs rather than embedding business logic locally.
- Feedback capture is built into the review experience so operational learning can be recorded at the point of use.

Accessibility Considerations

The supplied frontend context also points to a need for accessibility to be handled as part of normal delivery rather than as a later bolt-on activity. A practical approach for this application would include:

- linting and automated checks for common accessibility issues
- semantic HTML for interactive elements and page structure

- keyboard accessibility for navigation, forms, dialogs, and popups
- correct labelling and accessible error messaging for form interactions
- accessibility review as part of testing and code review

For this kind of claims application, accessible forms, visible focus states, and good keyboard behaviour are especially important because the user experience includes tables, dialogs, filters, and repeated data-entry or feedback interactions.

Operational Notes

- User experience quality depends on API responsiveness and the availability of persisted workflow outputs.
- Certain views are only meaningful when ingestion and orchestration have already run for the claim.
- The exact visual and layout details may evolve independently from the core backend behaviours documented elsewhere in the wiki.
- Frontend environments are available separately for development, testing, and UAT.
- Release and deployment are handled through Azure DevOps pipelines and container-based deployment.

Release And Deployment

- Repository: [Canopius.Apps.Claims.AugmentedTriage](#)
- Build and deployment model: Azure DevOps pipelines
- Rollback approach: redeploy the last known good Docker image tag through the deployment pipeline

This page focuses on the user-facing frontend footprint. The broader build, deployment, and promotion model is described in the deployment documentation.